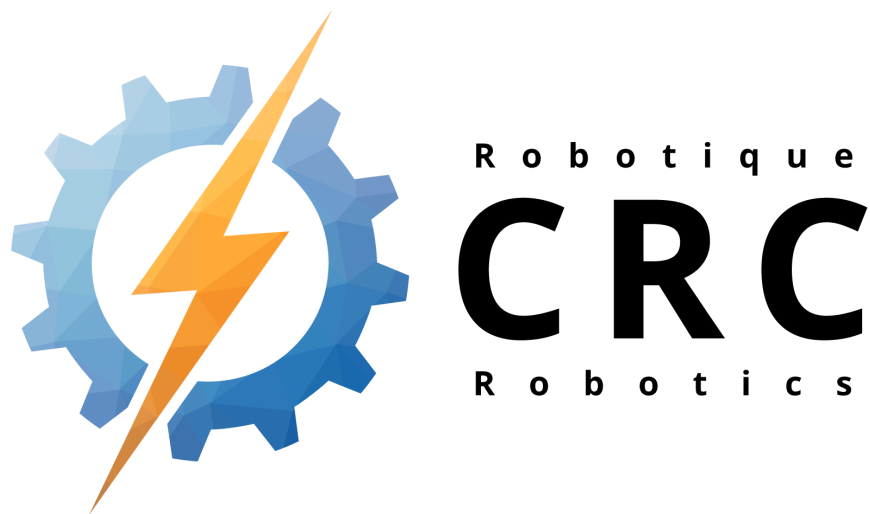




CRC
**PROGRAMMING
COMPETITION**



**PROBLEM BOOKLET
BLOCK B**

A FEW NOTES

- The complete rules are in section 4 of the rulebook. **Don't forget about multipliers!**
- You have until **19h59** to submit your solutions. Please submit the full folder in the same format as it was given to you with your solutions in the .py files.
- Feel free to ask organizers questions and discuss the problem with your team members. It is not an exam!
- **We are giving you quick and easy-to-use template files. You have to use them! We are also using templates to correct your solutions!**

USING THE TEMPLATE FILE

- All the files contain a function called `solve()` where you have to put your code. **DO NOT CHANGE THE NAME OF THAT FUNCTION!**
- To run the tests you will have to open a terminal (we strongly recommend to use `vscode`) and run the command `pytest`. If the command is ran in the folder containing all the problems, all tests are going to be ran
- To run a specific test, run it in the folder containing the test you want. You can do the same for sections, just navigate in the folder before executing the command `pytest`

STRUCTURE

Every problem contains a small introduction like this about the basics of the problem and what is required to solve it. Points distribution is also given here for the problems.

Input and output specification:

In these two sections, we specify what the inputs will be and what form they will take, and we also say what outputs are required for the code to produce and in what format they shall be.

Sample input and output:

In these two sections, you will find a sample input (that often has multiple entries itself) in the sample input and what your program should produce for such an input in the sample output.

First output explanation:

Sometimes, the problem might still be hard to understand after those sections, which is why there will also be a usually brief explanation of the logic that was used to reach the first output from the first input.

Table of contents

Trains & Metros (TM)	4
TM1: Tramway (30 points)	4
TM2: Metro ride! (15 points)	6
TM3: Loop Line (25 points)	7
TM4: Who has no ticket?? (111 points)	9
Mythology (MY)	11
MY1: Charybdis (20 points)	11
MY2: Thor (35 points)	13
MY3: Ariadne thread (70 points)	16
MY4: Return to Ithaca (90 points)	19
Winter Olympic (WO)	23
WO1: Speed skating (25 points)	23
WO2: Bobsleigh (45 points)	25
WO3: Ski jump (15 points)	27
WO4: You need to be the best to win a medal? Right? (30 points)	29
WO5: Perfect attacking lineup (55 points)	32

Trains & Metros (TM)

TM1: Tramway (30 points)

During a trip abroad, you must use the local tram system to get around. You will be given as input a list of strings representing the tram network in the form of a tree. You must then return a string containing the moves required to navigate the tree to reach your destination. The trees may vary in size from one example to another, but the row of possible destinations is always completely filled. (See below for examples of trees.)

For each junction you encounter, you must add to the output string either “L” (for Left) or “R” (for Right), depending on the direction you take as you move down the tree. **The left/right directions are given from your perspective as you view the tree; we do not use directions based on the tram’s point of view.**

Input Specification:

You will receive:

- **network:** an *array* containing *strings* that are the tram lines,
- **destination:** an *int* that is the destination to reach

Output Specification:

You will need to output:

- a *string* of the directions (“R”, “L”) to take to get to the target.

Sample input:

```
network=  ["      /\      ",
           "    /  \    ",
           "  /    \  ",
           "/\    /\ ",
           "6  3  4  7"]
destination=3
```

```

network=  ["          /\          ",
            "        /  \        ",
            "       /    \       ",
            "      /      \      ",
            "     /        \     ",
            "    /\        /\    ",
            "   /\      /\  /\   ",
            "  /\    /\ /\ /\ /\ ",
            "6 3 4 7 9 0 2 1"]

```

destination=0

```

network=  ["          /\          ",
            "        /  \        ",
            "       /    \       ",
            "      /      \      ",
            "     /        \     ",
            "    /\        /\    ",
            "   /\      /\  /\   ",
            "  /\    /\ /\ /\ /\ ",
            "6 8 5 2 3 0 1 7"]

```

destination=1

Sample output:

"LR"

"RLR"

"RRL"

First output explanation:

Here's the tree of the first example:

```

["          /\          ",
 "        /  \        ",
 "       /    \       ",
 "      /      \      ",
 "     /\        /\    ",
 "    6  3  4  7"]

```

We must reach destination 3, and we will need to pass through 2 junctions to get there. Therefore, we first go down the left side of the tree at the top, then the right side → "LR".

TM2: Metro ride! (15 points)

Schools must travel to the competition to participate! You must determine how long it takes the school to travel to the competition!

A train passes through each station every 5 minutes from the moment the trip starts, and the train takes 2.5 minutes to travel between stations.

Input Specification:

You will receive:

- ***Stations:*** An *array* of *int* of the amount of stations for each line. The starting station is included for every line

Output Specification:

You will need to output:

- A *float* of the amount of time it takes to get to the competition!

Sample input:

```
[8, 17]  
[27]  
[10, 10]
```

Sample output:

```
60.0  
65.0  
47.5
```

First output explanation:

In the first example, there are 7 stations on the first line. That takes 17.5 minutes, but you have to wait 2.5 minutes for the train on the next line as it only passes on every multiple of 5 minutes. Then, there are 16 stations to do that take 40 minutes which sums up to 60.0 minutes.

TM3: Loop Line (25 points)

You are responsible for designing the metro system for a city. You first choose to build all the stations, and then connect them with appropriate lines. While doing that, you realized that you can connect N stations in the system with a loop, so the train will depart from station 1, and then go to station 2, station 3, ..., until it reaches station N , and then it will return to station 1 again.

During a test operation of the track, you got bored and decided to take a guess on whether your train goes clockwise or counterclockwise on the loop. You decided to challenge yourself by not looking at the map. Luckily, you have a compass on your phone, and you designed a signaling system every kilometer.

After the train departs from station 1, every time you see a signal, you will check the compass and record the direction. You will repeat this process until you arrive at station 1 again. You will have a list of directions that you recorded, and you want to determine whether the train goes clockwise or counterclockwise.

Input Specification:

You will receive:

- ***directions***, a *string*, which is a sequence of characters N, E, S and W, representing the directions you recorded at each signal.

Output Specification:

You will need to output:

- a single *string*, which is either "CW" (if the train goes clockwise) or "CCW" (if the train goes counterclockwise).

Sample input:

NNEESSWWW

SSSEENWNENWW

WSWNWWNNESEENWWNNESENNEESWWSEESWWSEESWWSESWWNN

Sample output:

CW

CCW

CW

First output explanation:

With @ denoting the location of station 1 and * denoting the track between signals, the train's path can be visualized as follows:

```
* > * > * > *  
^           v  
*           *  
^           v  
*           *  
@ < * < * < *
```

and it is trivial to see that the train goes clockwise.

TM4: Who has no ticket?? (111 points)

You are a controller for the Montreal metro system and you want to know which station is associated with the “None” transit ticket. You have some information about the stereotypes of the different metro stations, and you will need to determine which one is associated with the most common transit pass being “None.” Take the information you are given and organize it into a table to recover all the details about the stations. In our example, the five stations are considered neighbors from left (station0) to right (station4).

You must fill in a 2D matrix containing the following information: the position from 0 to 4, the station’s color, the station’s name, the most common bag, the most common drink, and most importantly, the most frequently purchased transit ticket! Using a series of clues, you will need to reconstruct the 2D matrix and provide it to us.

The entries are in a completely random order, but here are the possible choices!

Line: Bleu, Vert, Orange, Jaune, Rose

Station: Snowdon, Berri-UQAM, Vendôme, Licorne, Atwater

Bag: Sac à dos, Tote bag, Sac à main, Bandoulière, Sac plastique

Drink: Tea, Jus, Coffee, Eau, Lait

Fare: Weekend illimité, Soirée illimité, Aucun, 1 passage, Mensuel

Input specification:

For input, you will receive a series of instructions as strings of one of 2 types:

- element 1 = element 2 (both elements belong to the same station)
- element 1 -> element 2 (element 1 is on the station left to the element 2)

Output specification:

As output, you must provide a 2D matrix filled with strings. You should list, in order, Station0 through Station4, followed by the station’s line, the station name, the stereotypical bag, the stereotypical drink, and the stereotypical fare. This applies to each station, starting from the station at position 0 up to the station at position 4.

ex: [[“Station0”, Line0, Name0, Bag0, Drink0, Fare0], [“Station1”, Line1, ...], ...]

The input:

Vendôme = Orange

Licorne = Soirée illimité

Snowdon = Sac à dos

Vert -> Rose

Vert = Coffee

Sac à main = Mensuel

Jaune = Bandoulière
Station2 = Lait
Station0 = Berri-UQAM
Weekend-illimité -> Sac à dos
Bandoulière -> 1 passage
Sac plastique = Jus
Atwater = Tote bag
Berri-UQAM -> Bleu
Eau -> Sac à dos

Sample output:

An array of 5 elements (stations) with each having 6 parts ("StationX", Line, Name, Bag, Drink, Fare) You have to find the output! (Yes there is a unique valid solution)

	Station0	Station1	Station2	Station3	Station4
Line					
Name					
Bag					
Drink					
Fare					

First output explanation:

For more details ask Frank.

Mythology (MY)

MY1: Charybdis (20 points)

Charybdis is a monster present in the Odyssey. She creates a whirlpool and resisting it is vital or the end will be near. Find where it is safe to sail.

She will be represented by the shape of a donut. The radius of the big outer circle will always be 1 (the center is always at (0,0)). The radius of the small inner circle will change each time. If you are within 0.15 of the inner circle, you are considered in the whirlpool. Find how many of the given spots are safe.

If a spot is exactly on the limit of the whirlpool, it is absorbed.

Note : the spots can be outside of the outer circle

Formula for a circle:

$$x^2 + y^2 = r^2$$

Input Specification:

You will receive:

- ***whirlpool***: a *float* representing the radius of the whirlpool
- ***positions***: an *array of tuple of float* for the positions of the ships

Output Specification:

You will need to output:

An *int* of the amount of positions that are safe

Sample input:

```
whirlpool = 0.45
```

```
positions = [(-0.3, 0.5), (0.25, -0.4), (1, -0.75)]
```

```
whirlpool = 0.24
```

```
positions = [(-0.1, -0.05), (0.3, -0.2), (0.0, 0.39), (0.1, -0.23)]
```

```
whirlpool = 0.84
```

```
positions = [(-0.45, 0.45), (0.31, -0.15), (0.5, -0.5), (-0.71, -0.7), (75.5, 3.45), (0.99, 0.01)]
```

Sample output:

1
0
3

First output explanation:

In the first example, the whirlpool has a radius of 0.45, so the radius of what is absorbed in the whirlpool is 0.6. The position (-0.3; 0.5) has a radius of approximately 0.58 which means it's not safe. The position (0.25; -0.4) has a radius of approximately 0.47 and is not safe. The 3rd position is at (1; -0.75) and has a radius of 1.25; it is safe. Therefore, there is only one safe spot.

MY2: Thor (35 points)

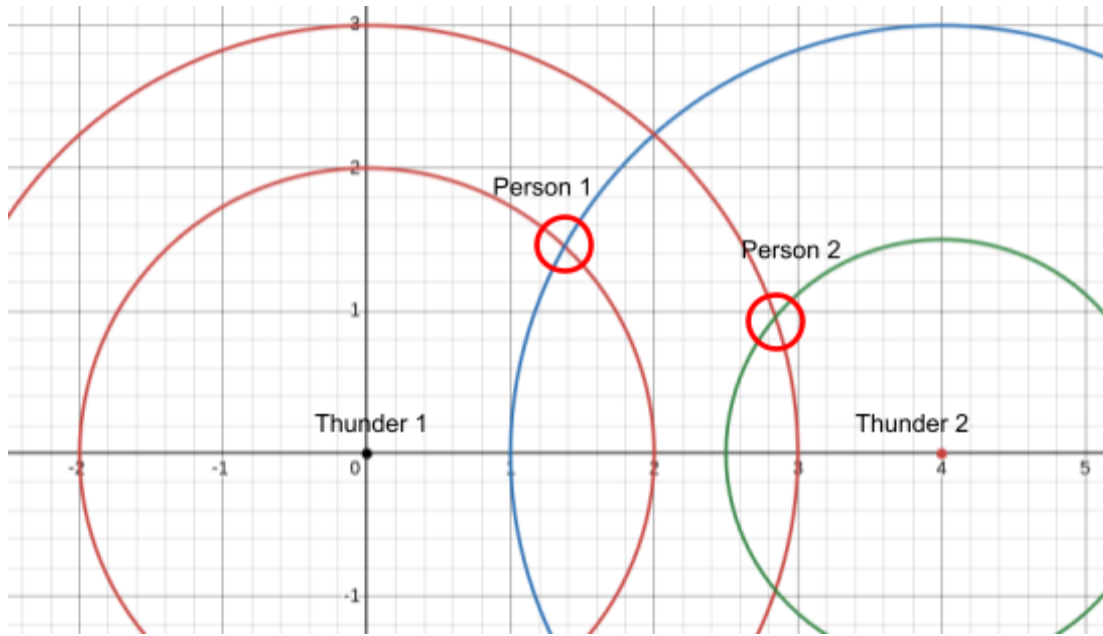
In Norse mythology, Thor is the god of thunder and lightning. His hammer, Mjolnir, is recognized as the cause of lightning. Lightning bolts are therefore said to be caused by Thor.

When lightning strikes, you have probably noticed that you see the flash well before you hear the sound. If we know the time difference between the moment we see the lightning and when we hear it, we can determine how far away the lightning has occurred.

There are two elements to consider in this problem: the time it takes for the light from the lightning to reach us and the time it takes for the sound to reach us. First, light travels at 300,000 km/s, while sound travels at about 340 m/s in the air. Since light travels extremely fast, we can consider the time it takes to reach us as instantaneous. Therefore, using the time difference between the flash and the thunder, we can determine our distance from the lightning by multiplying the time delay by the speed of sound.

If we have two people who both see and hear the same two lightning strikes, and who know precisely the time between the flash and the thunder, it is possible to triangulate their positions to determine the exact distance between the two people.

In short, if we have one lightning strike at a certain location and a second one exactly 4 km farther away, and if both people can determine their distance from each strike, they can determine the distance between them. One final assumption must be added to obtain a unique answer. Since being at a given distance from a point allows for two possible locations for each person, we must choose the option where both are on the same side of the center and therefore as close as possible to each other.



Input Specification:

You will receive:

- ***thunder_1***: A *tuple* of *floats* representing the time in seconds between the lightning flash and the sound of thunder for the two people relative to the first lightning strike (0,0).
- ***thunder_2***: A *tuple* of *floats* representing the time in seconds between the lightning flash and the sound of thunder for the two people relative to the second lightning strike (4,0).

Output Specification:

You will need to output:

- a *float* of the distance between the 2 observers (smallest possible distance).

Sample input:

```
thunder_1 = (5.88, 8.82)
```

```
thunder_2 = (8.82, 4.41)
```

```
thunder_1 = (2000/340, 2100/340)
```

```
thunder_2 = (2500/340, 2400/340)
```

```
thunder_1 = (1000/340, 2000/340)
```

```
thunder_2 = (3000/340, 2000/340)
```

Sample output:

1549.45
112.62
1000.00

First output explanation:

In the first example, person 1 hears the first lightning strike 5.88 seconds after seeing it, and person 2 hears it 8.82 seconds after seeing it. From this, since the speed of sound is 340 m/s in this problem, we obtain a distance of:

$$d_{1,1} = 340 * 5.88 = 1999.2 \text{ m}$$

$$d_{1,2} = 340 * 8.82 = 2998.8 \text{ m}$$

We can now do the same to find the distance of the 2 observers relative to the second lightning:

$$d_{2,1} = 340 * 8.82 = 2998.8 \text{ m}$$

$$d_{2,2} = 340 * 4.41 = 1499.4 \text{ m}$$

We now have the distance between each of the two people and each of the two lightning strikes, and we are also given in the problem that the distance between the two lightning strikes is exactly 4000 m.

Using a few circle equations (which I am providing to simplify things for you), here are the x and y coordinates based on the distances from the two circles.

$$x = (d_{e1}^2 - d_{e2}^2 + 4000^2) / (2 * 4000)$$

$$y = \sqrt{d_{e1}^2 - x^2}$$

We can therefore determine the x and y positions of our two observers. This allows us to find the distance between them using the 2D distance formula:

$$distance = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

MY3: Ariadne thread (70 points)

In Greek mythology, Ariadne gives Theseus a thread to help him find his way out of the labyrinth after defeating the Minotaur. The idea is simple: as you enter the labyrinth, you unwind a thread so that you can retrace your steps and find the exit.

You enter the labyrinth through a single entrance, must reach a target, and then exit through the same entrance. The labyrinth contains multiple possible paths, intersections, and dead ends. Unlike a simple corridor, it is sometimes impossible to avoid passing through certain cells more than once.

Your goal is to determine the minimum number of cells that must necessarily be visited twice: once while entering the labyrinth and reaching the target, and a second time while returning to the original entrance. The entrance cell E and the target cell O do not count as being visited twice.

We assume that:

- you always choose an optimal path,
- you have perfect knowledge of the maze,
- you may take any open path,
- but you cannot pass through walls.

The maze is represented as an array of strings:

- # represents a wall
- . represents an open cell
- E represents the entrance of the maze (only one)
- O represents the objective to reach (only one)

The entrance E is also the exit: you must return to it after reaching O.

Movement is allowed only in the four cardinal directions: up, down, left, right.

Input Specification:

You will receive:

- ***labyrinth***: a 2D array of *strings* of the labyrinth

Output Specification:

You will need to output:

- an *int* that is the minimal amount of squares that will have to be visited twice to go from E to O and back.

Sample input:

```
[
  "#####",
  "#E...#",
  "#.#.#.#",
  "#.#...#",
  "#.###.#O.#",
  "#.....#",
  "#.#####",
  "#####"
]
[
  "#####",
  "#E.....#",
  "#.###.#.#.###.#",
  "#...#.#.#...#.#",
  "###.#.#.###.#.#",
  "#...#.....O.#.#",
  "#.#####.#####.#",
  "#####"
]
[
  "#####",
  "#E...#",
  "#.#.#.#O#",
  "#.#.....#",
  "#####"
]
```

Sample output:

```
8
4
6
```

First output explanation:

Here's a first way to get from the start of the maze to the end of the maze:

```
[
"#####",
"#E...#...#",
"#.#.#.#.#",
"#.#...#...#",
"#.###.#O.#",
"#.....#...#",
"#.#####...#",
"#####"]
```

Here's the best way to come back the to start using as few squares that were already used:

```
[
"#####",
"#E...#...#",
"#.#.#.#.#",
"#.#...#.#",
"#.###.#O.#",
"#.....#...#",
"#.#####...#",
"#####"]
```

Here are the 8 squares that absolutely need to be crossed twice.

```
[
"#####",
"#E...#...#",
"#.#.#.#.#",
"#.#...#.#",
"#.###.#O.#",
"#.....#...#",
"#.#####...#",
"#####"]
```

MY4: Return to Ithaca (90 points)

The *Iliad* and the *Odyssey* are the two great epics of the Greek poet Homer and are among the most famous stories in Greek mythology. What some may not know is that the second is a direct sequel to the first and tells the story of the 10-year journey that Odysseus, king of Ithaca, endured to return home from the Trojan War. The first problem in this category focuses specifically on one of the adventures he faced—Charybdis. Here, we are primarily concerned with Odysseus' sea voyage itself.

During many parts of his journey, gods displeased by his arrogance unleash storms and strong winds on Odysseus' fleet, and he ends up stranded with his men on several islands before finally reaching home. **Given a specific archipelago of islands (the same for all examples), an initial speed, and a series of winds, you will need to calculate the total duration of Odysseus' voyage in hours.**

(See explanation for a diagram) Odysseus will start from the point (0,0) at the bottom-left of a Cartesian plane and must reach the island of Ithaca, which is represented as a square bounded by the points (90,100), (100,100), (100,90), and (90,90). The boat will never leave the region defined by $0 \leq x \leq 100$ and $0 \leq y \leq 100$. Distances are given in kilometers, and speeds in km/h. To determine the boat's movement after a time "t" given an initial speed " v_0 ", an initial position " x_0 ", and a wind "w," we define the force acting on the boat:

$$\vec{F} = mk(\vec{w} - \vec{v})$$

Don't worry: we have already done the necessary calculations to obtain the equations of motion for you! **You will need to calculate the positions and velocities in the x-direction independently from the y-direction, or you will encounter difficulties.** The boat's position after a time "t" under a constant wind can be determined using the following equations:

$$x(t) = x_0 + w_x t - 60(w_x - v_{0x})(1 - e^{-t/60})$$

$$y(t) = y_0 + w_y t - 60(w_y - v_{0y})(1 - e^{-t/60})$$

where "t" is in minutes. (x_0, y_0) is the initial position of the movement. (v_{0x}, v_{0y}) is the initial speed, and (w_x, w_y) is the given wind. It may be useful to remember that, for a vector of magnitude "z" at a given angle "θ," the vector components are given by ($z \cos \theta, z \sin \theta$). For example, Odysseus' initial speed at the start of the problem is given by ($v_0 \cos 45^\circ, v_0 \sin 45^\circ$), as he always begins heading northeast. **The winds will also be given in cardinal directions, and it will be up to you to extrapolate their components.** You will also need the equations to calculate the boat's speed at time "t" **in the event of a wind change at sea: (in this case, your final speed at the end of a constant wind period becomes the initial speed for the next constant wind period):**

$$v_x(t) = w_x - (w_x - v_{0x})e^{-t/60}$$

$$v_y(t) = w_y - (w_y - v_{0y})e^{-t/60}$$

Finally, while it is “straightforward” to piece together trajectories to calculate the time required to reach Ithaca, you must also take into account other islands in the archipelago that may affect the trajectory. **If the boat collides with an island, it becomes stranded and cannot depart until a favorable wind occurs;** that is, if the boat is stranded on the north coast of an island, for example, it will require a wind with a northward component to set off again (i.e., “NW,” “N,” or “NE”). **The time spent waiting for such a wind must be counted in the total voyage duration. The islands are rectangles defined by 4 points on the Cartesian plane. The boat departs from a stranded island at rest (its initial speed is zero).** The islands, which are constant across examples, can be copied here:

```
islands=[[ [10,32],[30,32],[30,50],[10,50]],
           [63,15],[94,15],[94,31],[63,31]],
          [3,76],[24,76],[24,95],[3,95]],
          [35,2],[57,2],[57,9],[35,9]],
          [47,61],[79,61],[79,82],[47,82]]]
```

Odysseus’ voyage will stop once the boat reaches land on the island of Ithaca:

```
ithaca=[[90,90],[100,90],[100,100],[90,100]]
```

You will need to check for a collision with Ithaca or any other island at every **MINUTE of the voyage. The formulas are given in minutes, and the examples were calculated using the minute as the base unit: if you use anything other than minutes, you will get a different result for the total voyage duration!**

Input Specification:

You will receive:

- ***v0***: an *int* that represents the initial speed in km/h of Odysseus’ boat towards the North-East
- ***winds***: a 2D *array* of the sequence of the winds that will affect Odysseus’ boat in chronological order. Each wind is described as follows: [***force***, ***direction***, ***duration***]. ***force*** is an *int* corresponding to the speed of the wind in km/h. ***direction*** is a *string* of 1 or 2 characters that represents a cardinal direction (“N”, “E”, “S”, “O”) or subcardinal direction (“NE”, “SE”, “SO”, “NO”) in which the wind blows. Finally, ***duration*** is an *int* that indicates the duration of the wind in hours.

Output Specification:

You will need to output:

- A *float* rounded to the hundredth of the total hours that have passed since the beginning of the voyage (since the start at (0, 0)) until Odysseus arrives on the island of Ithaca.

Sample input:

1. 25, [[16, "E", 5], [17, "NE", 3], [11, "NO", 3], [29, "E", 2], [42, "N", 5], [12, "SO", 7]]
2. 47, [[12, "NE", 3], [7, "E", 5], [18, "N", 9]]
3. 5, [[55, "NE", 7], [31, "SE", 5], [60, "S", 3], [19, "N", 4], [37, "S", 7]]
4. 29, [[37, "E", 1], [15, "NO", 6], [15, "NE", 2], [23, "O", 2], [36, "E", 1], [31, "N", 1], [46, "E", 17]]

Sample output:

1. 15.05
2. 10.68
3. 19.13
4. 14.95

First output explanation:

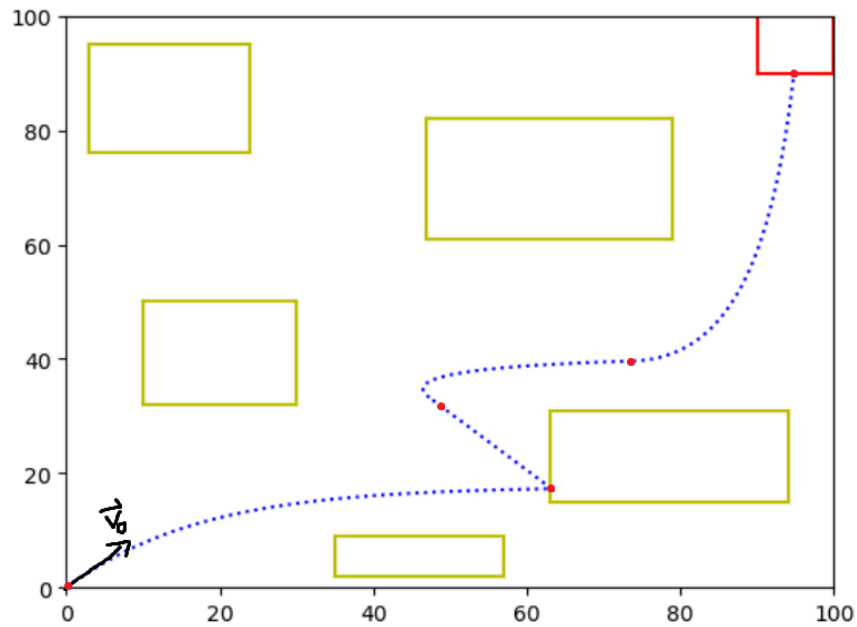
The first wind blows toward the East for 5 hours, causing the boat to run aground on the west beach of the rightmost yellow island according to our calculations.

After these 5 hours, the wind changes to a Northeast wind! Since the wind pushes the boat toward the land instead of the sea, it cannot make any progress, and Odysseus is stuck on this island for another 3 hours.

Finally, a Northwest wind rises at 11 km/h, allowing Odysseus and his companions to slowly depart from where they were stranded and make some progress at sea. As there are no islands encountered under this wind, they are still in open sea when the wind changes, so we must calculate the boat's speed at the end of these three hours to use as the initial speed for the next wind.

Next, an East wind blows and completely changes the boat's trajectory. Since they are still at sea after 2 hours under this wind, we repeat the process.

Finally, a strong North wind allows Odysseus and his companions to reach Ithaca after an additional 2 hours and 3 minutes. Since they have arrived, the last wind in the list does not matter. The total voyage time was $5 + 3 + 3 + 2 + 2.05 = 15.05$ hours.



The yellow rectangles represent the islands at sea. The red rectangle is the island of Ithaca, which is the required destination. Odysseus starts from the point (0,0) with an initial speed heading Northeast. Each red point roughly corresponds to the start and/or end of a significant wind period during the journey. The blue dotted line represents the trajectory in question.

Winter Olympic (WO)

WO1: Speed skating (25 points)

The Winter Olympics have multiple sports like speed skating! You will need to determine who wins the race and with what time according to a few statistics.

The speeds are in km/h

Input Specification:

You will receive:

- ***distance***: an *int* representing the length of the race in meters
- ***skaters***: an *array* of *string* with the names of the skaters
- ***initial_speed***: an *array* of *float* that is the initial speed of the skaters until they run out of stamina
- ***stamina***: an *array* of *int* that indicates the amount of meters the skater can go with their initial speed
- ***second_speed***: an *array* of *float* that is a skater's speed after its stamina is depleted

An index always points to the same skater no matter the chosen statistics *array*
i=0 concerns the first skater in every *array*

Output Specification:

You will need to output:

- A *string* with the name of the winner followed by their time in seconds rounded to the hundredth

ex: "name, time.00s"

Sample input:

```
distance = 500
```

```
skaters = ["Serina", "Beatrice", "Jutta"]
```

```
initial_speed = [48.26, 53.35, 48.45]
```

```
stamina = [750, 400, 600]
```

```
second_speed = [47.23, 34.16, 38.45]
```

```
distance = 1000
```

```
skaters = ["Courtney", "Li", "Gilli", "Arianna"]
```

```
initial_speed = [44.10, 43.90, 43.36, 43.70]
```

```
stamina = [750, 750, 625, 625]
```

```
second_speed = [19.79, 19.36, 27.10, 26.86]
```

```
distance = 2400
skaters = ["United States", "Japan", "Canada", "Pays-Bas"]
initial_speed = [48.19, 48.41, 54.61, 49.03]
stamina = [1600, 1200, 2000, 1800]
second_speed = [46.10, 48.40, 48.93, 29.32]
```

Sample output:

```
"Jutta, 37.15s"
"Gilli, 101.71s"
"Canada, 161.27s"
```

First output explanation:

In the first input, there are 3 skaters: Serina has a time of 37.28s, Béatrice got a time of 26.99s at her highest speed and 10.54s at the slower pace, and Jutta got a time of 37.15s that is calculated using this formula

Time = distance/speed x 3.6

So doing the formula for her gives $500/48.45$ or, approximately, 10.3199 which sums up to a total time of 37.15s.

WO2: Bobsleigh (45 points)

Bobsleigh is one of the fastest sports at the Olympic Games, with speeds exceeding 130, 140, and sometimes even 150 km/h! At those speeds, every decision must be planned in advance, and the track must be known by heart.

You must write a program to help the athletes by analyzing the track and then providing instructions for each segment. If there is no change in direction, output **R**. If the track turns to the left, output the letter **G**. Finally, if it turns to the right, output the letter **D**.

Input Specification:

You will receive no input:

- **path**: an *array of strings* that is the path of the bobsleigh track

Output Specification:

You will need to output:

- a *string* that represents the direction to take, either R for not turning, G for turning left and D to turn right

Sample input:



Sample output:

RRRRGRRRRGRDRDRGRGRDRRRDRRDRGRDRGRGRDRDRRRRRRRDRRGRRRGRGRDRRR

First output explanation:

The descent starts at **D** and ends at **F**. When the bobsleigh is going straight, nothing needs to be done but, in order to avoid crashing the sled, you need to turn left (R) or right (D).

WO3: Ski jump (15 points)

During a ski jumping competition at the Winter Olympic Games, each athlete performs a jump that is evaluated according to two criteria:

- the distance of the jump
- the scores given by the judges

The final score of a jump is calculated according to the following rules:

1. Each meter of distance is worth **2 points**.
2. Five judges each give an integer score between **0 and 20**.
3. The **highest** and the **lowest** scores are removed.
4. The remaining **three scores** are added together.
5. The final score is the sum of:
 - the distance points
 - the retained judges' points.

Input Specification:

You will receive:

- **distance:** an *int* of the distance in meters jumped by the athlete
- **scores:** an *array* of *int* between 0 and 20 that are the scores of the 5 judges

Output Specification:

You will need to output:

- an *int* of total score for the jump

Sample input:

```
distance = 120
```

```
scores = [18, 19, 17, 20, 16]
```

```
distance = 95
```

```
scores = [15, 15, 16, 14, 15]
```

```
distance = 130
```

```
scores = [20, 20, 19, 18, 17]
```

Sample output:

294

235

317

First output explanation:

Distance points: $120 \times 2 = 240$

Sorted scores: [16, 17, 18, 19, 20]

Remove 16 and 20

Sum of remaining scores: $17 + 18 + 19 = 54$

Final score: $240 + 54 = 294$

WO4: You need to be the best to win a medal? Right? (30 points)

An Olympic tournament brings together 16 teams. Each team has an integer strength rating between 1 and 100.

The tournament is organized as a single-elimination bracket:

- 16 teams → 8 winners
- 8 teams → 4 winners
- 4 teams → 2 winners
- Final

The semifinalists (top 4) compete for medals:

- The two finalists receive gold and silver.
- The two other semifinalists compete for the bronze medal.

Therefore, a team wins a medal if it finishes in the top 3 (either reaches the final or wins the bronze match).

Rules:

- If two teams face each other, the team with the higher rating always wins.
- All ratings are different.
- You may freely arrange the bracket at the beginning of the tournament.
- You choose the initial placement of the 16 teams.

You must determine whether there exists a bracket arrangement in which the chosen team can win a medal.

Input Specification:

You will receive:

- **ratings** : an *array* of *int* that are the rating of each team
- **team**: an *int* of the position in the ratings of the chosen team

Output Specification:

You will need to output:

- an *int* of the best possible position with optimal seeding and if it is not top 3, the value 0.

Sample input:

[100, 95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25]

8

[95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25, 20]

5

[100, 95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25]

0

Sample output:

2

2

1

First output explanation:

In the first example, we want to arrange the bracket so that the team with a rating of 60 earns a medal. Eight teams are stronger and seven teams are weaker than our chosen team. Here is one possible arrangement to achieve the silver medal:

100 vs 95	60 vs 55
90 vs 85	50 vs 45
80 vs 75	25 vs 35
70 vs 65	40 vs 20

100 vs 90	60 vs 50
80 vs 70	35 vs 40

100 vs 80	60 vs 40
-----------	----------

Grand Final: 100 vs 60

3rd place match: 80 vs 40

Gold medal: 100

Silver medal: 60

Bronze medal: 80

We can see here that even without being the strongest team at all, there is still a way to hope for an Olympic medal! This time, our team with a rating of 60 earns the silver medal!

WO5: Perfect attacking lineup (55 points)

A hockey team typically dresses 12 forwards, 6 defensemen, and 2 goalies for a game. That means 4 lines of 3 forwards, 3 pairs of defensemen, with 1 goalie in net and another as backup.

Although much of the strategy happens on the ice, it is important to carefully assign each player to the right position in order to form the best possible lineup.

In this problem, you must form the best possible team by distributing the 12 forwards across the 4 offensive lines. Each forward has two statistics to consider: endurance and natural talent. Their efficiency score will depend on both their natural talent and their endurance, and it will vary depending on the line they are assigned to.

You must place:

- the 4 left wingers across the 4 offensive lines,
- the 4 centers across the 4 offensive lines,
- the 4 right wingers across the 4 offensive lines.

Your objective is to build the best offensive lineup in order to maximize total efficiency.

Players on the first line have the greatest impact on the game due to their strong presence. Their efficiency score is calculated as:

$$efficacy = talent * 1.2 + endurance * 1.2$$

Players on the second line are often very talented but play slightly fewer minutes. Their efficiency score is:

$$efficacy = talent * 1.2 + endurance$$

Players on the third line are usually slightly weaker than the top two lines. Their efficiency score is:

$$efficacy = talent + endurance$$

Players on the fourth and final line are often less talented, and their role is typically to bring energy and boost team morale. Their efficiency score is:

$$efficacy = talent * 0.6 + endurance * 1.2$$

You must output, in order, the 4 lines, listing the jersey number of the left winger, then the center, then the right winger for each offensive line, in order to maximize the team's total efficiency score.

Input Specification:

You will receive:

- ***left_wing***: an *array* of the 4 left wing players with their number followed by their talent score and then their endurance score
- ***center***: an *array* of the 4 center players with their number followed by their talent score and then their endurance score
- ***right_wing***: an *array* of the 4 right wing players with their number followed by their talent score and then their endurance score

Output Specification:

You will need to output:

- an *array of arrays* of *int* of the number of the players (left, center, right) for each line.

Sample input:

```
l_w = [(26, 8, 8), (98, 10, 9), (56, 7, 6), (44, 6, 7)]
```

```
c = [(23, 7, 7), (66, 6, 7), (8, 9, 10), (12, 8, 8)]
```

```
r_w = [(41, 4, 8), (74, 9, 9), (33, 6, 6), (55, 9, 10)]
```

```
l_w = [(11, 10, 5), (22, 5, 10), (33, 8, 8), (44, 3, 3)]
```

```
c = [(55, 9, 6), (66, 6, 9), (77, 7, 7), (88, 4, 4)]
```

```
r_w = [(91, 10, 10), (92, 2, 8), (93, 8, 2), (94, 5, 5)]
```

```
l_w = [(1, 7, 8), (2, 8, 7), (3, 6, 6), (4, 5, 5)]
```

```
c = [(5, 9, 8), (6, 8, 9), (7, 7, 7), (8, 6, 6)]
```

```
r_w = [(9, 10, 7), (10, 7, 10), (11, 8, 8), (12, 5, 6)]
```

Sample output:

```
[[98, 8, 55], [26, 12, 74], [56, 23, 33], [44, 66, 41]]
```

```
[[33, 66, 91], [11, 55, 93], [44, 77, 94], [22, 88, 92]]
```

```
[[1, 6, 10], [2, 5, 9], [3, 7, 11], [4, 8, 12]]
```

First output explanation:

Yes, this is the logic given by chat gpt (no you cannot use it to help you during this competition sorry) and it is accurate so here's chat gpt explanation...

Step 1: Understand the scoring per line

Each line has a different formula for efficiency:

Line	Formula
1st	$\text{talent} * 1.2 + \text{endurance} * 1.2$
2nd	$\text{talent} * 1.2 + \text{endurance}$
3rd	$\text{talent} + \text{endurance}$
4th	$\text{talent} * 0.6 + \text{endurance} * 1.2$

Observation:

- The **1st line** values talent and endurance the most (multiplied by 1.2).
- The **2nd line** favors talent more than endurance.
- The **3rd line** is balanced.
- The **4th line** favors endurance more than talent ($0.6 \times \text{talent}$, $1.2 \times \text{endurance}$).

So the strategy is:

- Put the **best talent+endurance** combination on the **1st line**.
- Put slightly less strong players on the 2nd and 3rd lines.
- Put players who may have low talent but good endurance on the 4th line.

Step 2: Compute the "effective" score for each player

For clarity, let's compute the **1st line scores** for all players:

Left wings (l_w) for 1st line formula: $\text{talent}1.2 + \text{endurance}1.2$

- $26 \rightarrow 81.2 + 81.2 = 9.6 + 9.6 = 19.2$
- $98 \rightarrow 101.2 + 91.2 = 12 + 10.8 = 22.8$ ✓
- $56 \rightarrow 71.2 + 61.2 = 8.4 + 7.2 = 15.6$
- $44 \rightarrow 61.2 + 71.2 = 7.2 + 8.4 = 15.6$

Centers (c) for 1st line:

- $23 \rightarrow 71.2 + 71.2 = 8.4 + 8.4 = 16.8$
- $66 \rightarrow 61.2 + 71.2 = 7.2 + 8.4 = 15.6$
- $8 \rightarrow 91.2 + 101.2 = 10.8 + 12 = 22.8$ ✓
- $12 \rightarrow 81.2 + 81.2 = 9.6 + 9.6 = 19.2$

Right wings (r_w) for 1st line:

- $41 \rightarrow 41.2 + 81.2 = 4.8 + 9.6 = 14.4$
- $74 \rightarrow 91.2 + 91.2 = 10.8 + 10.8 = 21.6$
- $33 \rightarrow 61.2 + 61.2 = 7.2 + 7.2 = 14.4$
- $55 \rightarrow 91.2 + 101.2 = 10.8 + 12 = 22.8$ ✓

Observation:

- **Best left wing:** 98
- **Best center:** 8
- **Best right wing:** 55

✓ That matches the **first line of the output**: [98, 8, 55]

Step 3: Assign the 2nd line

2nd line formula: talent*1.2 + endurance

We remove the first-line players:

- Remaining left wings: 26, 56, 44
- Remaining centers: 23, 66, 12
- Remaining right wings: 41, 74, 33

Compute approximate scores:

Left wings:

- $26 \rightarrow 8*1.2 + 8 = 9.6 + 8 = 17.6$ ✓
- $56 \rightarrow 7*1.2 + 6 = 8.4 + 6 = 14.4$
- $44 \rightarrow 6*1.2 + 7 = 7.2 + 7 = 14.2$

Centers:

- $23 \rightarrow 7*1.2 + 7 = 8.4 + 7 = 15.4$
- $66 \rightarrow 6*1.2 + 7 = 7.2 + 7 = 14.2$
- $12 \rightarrow 8*1.2 + 8 = 9.6 + 8 = 17.6$ ✓

Right wings:

- $41 \rightarrow 4 \cdot 1.2 + 8 = 4.8 + 8 = 12.8$
- $74 \rightarrow 9 \cdot 1.2 + 9 = 10.8 + 9 = 19.8$ ✓
- $33 \rightarrow 6 \cdot 1.2 + 6 = 7.2 + 6 = 13.2$

Best combination for line 2: [26, 12, 74] ✓ Matches output.

Step 4: Assign the 3rd line

3rd line formula: talent + endurance

Remaining players:

- Left wings: 56, 44
- Centers: 23, 66
- Right wings: 41, 33

Compute sums:

- Left: $56 \rightarrow 7+6=13$, $44 \rightarrow 6+7=13$
- Centers: $23 \rightarrow 7+7=14$, $66 \rightarrow 6+7=13$
- Right wings: $41 \rightarrow 4+8=12$, $33 \rightarrow 6+6=12$

Best total combination:

- Left: 56
- Center: 23
- Right: 33

✓ Matches output: [56, 23, 33]

Step 5: Assign the 4th line

Remaining players: [44, 66, 41]

- Line 4 formula: talent 0.6 + endurance 1.2
- Left: $44 \rightarrow 6 \cdot 0.6 + 7 \cdot 1.2 = 3.6 + 8.4 = 12$
- Center: $66 \rightarrow 6 \cdot 0.6 + 7 \cdot 1.2 = 3.6 + 8.4 = 12$
- Right: $41 \rightarrow 4 \cdot 0.6 + 8 \cdot 1.2 = 2.4 + 9.6 = 12$

✓ Matches output: [44, 66, 41]

✓ Step 6: Summary

- **Line 1 (starters):** [98, 8, 55] → best talent + endurance
- **Line 2:** [26, 12, 74] → next strongest combination
- **Line 3:** [56, 23, 33] → balanced medium line
- **Line 4 (energy/morale line):** [44, 66, 41] → weaker players, focus on endurance